

Student Maria Camila Garcia Ramirez
Teacher William David Prada Buitrago
Subject Introducción a la visión por computador

Proyecto de Visión por Computadora:

Asistente Basado en Detección de Obstáculos Urbanos para Personas con Discapacidad Visual en Bogotá

1 Introducción

La movilidad urbana representa un desafío significativo para personas con discapacidad visual, especialmente en ciudades como Bogotá donde el estado de los andenes, la presencia de huecos, basuras, postes, vehículos y semáforos no adaptados incrementan el riesgo en el desplazamiento diario.

Este proyecto desarrolla **Urban Assistant – Camino Seguro**, un sistema de asistencia basado en visión por computador que opera en tiempo real y tiene como propósito brindar alertas auditivas sobre obstáculos urbanos cercanos. El sistema combina tres componentes principales:

- **YOLOv8 exportado a TensorFlow Lite** para detección eficiente de objetos en CPU.
- **MiDaS Small** para estimación de profundidad monocular y cálculo de distancia relativa.
- **Sistema de alertas por voz** implementado con pyttsx3 y lógica de priorización.

El objetivo final es proporcionar un asistente urbano accesible, ligero y capaz de funcionar sin GPU, preparando el terreno para una versión móvil en Android. Este documento describe la arquitectura adoptada, las decisiones técnicas, la metodología de integración y los resultados preliminares del prototipo funcional.

2 Objetivos

2.1 Objetivo General

Desarrollar un sistema de visión por computador capaz de detectar objetos urbanos y estimar su proximidad en tiempo real para emitir alertas auditivas dirigidas a personas con discapacidad visual.

2.2 Objetivos Específicos

- Entrenar un modelo YOLOv8 con un dataset propio de 10 clases urbanas relevantes.
- Exportar el modelo de detección a TensorFlow Lite para ejecución eficiente en CPU.
- Integrar un estimador de profundidad monocular (MiDaS Small) que permita calcular distancias aproximadas.
- Implementar un sistema de alertas por voz con lógica de debounce y umbrales críticos.
- Diseñar un pipeline modular que combine detección, profundidad, NMS y fusión de distancias.
- Realizar pruebas en entorno real con cámara RGB para validar utilidad y estabilidad del sistema.

3 Estado del Arte

La asistencia tecnológica para personas con discapacidad visual ha pasado de dispositivos simples a sistemas avanzados que integran visión por computadora y aprendizaje profundo.

1. **Bastones inteligentes:** como WeWALK, que integran sensores ultrasónicos y asistencia por voz. Útiles a corta distancia (0.8–2.5 m), pero sin detección semántica de objetos.
2. **Aplicaciones móviles:** como Lazarillo App, que combina GPS, mapas y audio para orientación. Su limitación es la dependencia de cobertura satelital y la falta de detección dinámica de obstáculos.
3. **Visión por computadora:** modelos como YOLOv5/YOLOv8 permiten detección en tiempo real (>30 FPS). Ejemplo: Guardian Angel (2022) con YOLOv3 alcanzó $mAP@0.5 \approx 78\%$. Algunos trabajos ya incluyen detección de semáforos y señales.
4. **Estimación de profundidad:** modelos como MiDaS generan mapas densos y, al fusionarse con YOLO, permiten asociar distancias a objetos y emitir alertas contextuales (ej. “hueco a 1.1 m”).
5. **Infraestructura urbana inteligente:** semáforos accesibles en España y Brasil muestran potencial, pero requieren infraestructura específica, lo que limita su escalabilidad.
6. **Datasets públicos relevantes**
 - (a) **COCO (Common Objects in Context)**
 - (b) **KITTI Vision Benchmark Suite**
 - (c) **Mapillary Vistas Dataset**
 - (d) **Open Images Dataset (Google)**

4 Metodología

El sistema fue implementado siguiendo una arquitectura modular en Python, con énfasis en ligereza, eficiencia y capacidad de ejecución en tiempo real sin GPU. Cada módulo corresponde a una etapa del pipeline completo.

4.1 Captura y Preprocesamiento

Se utiliza la cámara principal del computador mediante OpenCV. Cada frame se redimensiona a 640×640 píxeles, se normaliza en el rango $[0,1]$ y se convierte al formato esperado por el modelo TensorFlow Lite. Este preprocesamiento garantiza coherencia con el modelo entrenado y un consumo óptimo de memoria.

4.2 Detección de Objetos con YOLOv8-TFLite

El modelo de detección se entrenó sobre el dataset personalizado **proyecto-ubaei**, compuesto por 10 clases relevantes en el contexto urbano de Bogotá: Andén, Basura, Dog, Hueco, Objetos, Persona, Poste, Semáforo, Silla y Stop.

Posteriormente, se exportó el modelo final como `best_float32.tflite`, permitiendo ejecución eficiente en CPU. La salida del modelo presenta 8400 predicciones por frame; por esto se implementó un postprocesamiento manual consistente en:

1. Aplicación de la función sigmoide sobre los puntajes de clase.
2. Filtrado inicial por umbral de confianza.
3. Filtrado por área mínima para evitar detecciones espurias.
4. **Non-Max Suppression (NMS)** por clase, reduciendo redundancias.
5. Restricción a máximo 2 detecciones por clase y 10 por frame.

Este proceso reduce ruido, mejora precisión y mantiene un rendimiento adecuado en tiempo real.

4.3 Estimación de Profundidad con MiDaS Small

MiDaS Small se integró mediante `torch.hub` debido a su eficiencia computacional y a su capacidad para ejecutarse en CPU. El modelo produce un mapa de profundidad relativa, que se normaliza para extraer la mediana del área correspondiente a cada *bounding box*, generando así una estimación aproximada de la distancia al objeto.

La distancia se calcula con la expresión:

$$D = \frac{1}{\text{mediana} \cdot k}$$

donde k es un factor de calibración determinado experimentalmente en pruebas reales con objetos a distancias conocidas.

4.4 Lógica de Alertas

Para cada objeto detectado se evalúa su distancia (si está disponible) o la proporción del objeto en pantalla (*box ratio*), utilizada como respaldo para casos donde MiDaS presenta inconsistencias. Las alertas son emitidas mediante `pyttsx3`, con un mecanismo de **debounce de 0.8 s** para evitar repetición excesiva.

Las alertas se formulan de la siguiente manera:

- “Clase a X metros” cuando se dispone de una estimación confiable de MiDaS.
- “Clase muy cerca” cuando el *box ratio* supera los umbrales establecidos.

4.5 Flujo General del Sistema

Cámara → YOLO TFLite → Postprocesamiento + NMS → MiDaS
→ Cálculo de distancia → Alertas por voz → Usuario

4.6 Dataset y Balance de Clases

Para el desarrollo se empleó el dataset personalizado **proyecto-ubaei**, construido en Roboflow. A diferencia de COCO o KITTI, este dataset incluye clases críticas del entorno urbano de Bogotá, como *Hueco*, *Andén* o *Basura*.

Las 10 clases incluidas fueron:

Andén, Basura, Dog, Hueco, Objetos, Persona, Poste, Semáforo, Silla, Stop.

4.6.1 Características del Dataset

- **Fuente de datos:** Imágenes propias capturadas en Bogotá, complementadas con imágenes web para clases de baja frecuencia como *Hueco* o *Basura*.
- **Anotación:** Todas las imágenes fueron etiquetadas con *bounding boxes* en formato YOLO.
- **Balance:** Se aplicó *data augmentation* para reducir la desproporción entre clases (p.ej., Persona y Andén poseen más ejemplos que otras clases).
- **División:** 70% entrenamiento, 20% validación y 10% prueba.

Este dataset permitió entrenar un modelo adecuado a las condiciones reales de Bogotá.

4.7 Cálculo de Distancia

La estimación de distancia se basó en un enfoque híbrido basado en tres métodos, aunque el prototipo final empleó principalmente MiDaS Small:

- **Método 1: MiDaS (implementado)**
Produce profundidad relativa por píxel. Se toma la mediana del *bounding box* y se convierte a distancia mediante calibración con un factor k .

- **Método 2: Modelo pinhole (propuesto, no implementado)**

Estimaría distancia usando altura real conocida:

$$Z = \frac{f \cdot H_{\text{real}}}{h_{\text{pixels}}}$$

Este método se descartó por su baja robustez ante variaciones de pose.

- **Método 3: Fusión MiDaS + pinhole (propuesto)**

Su implementación fue evaluada, pero se priorizó la simplicidad computacional.

4.8 Avisos y Prioridad

El sistema prioriza objetos según riesgo:

- **Postes, sillas, basuras:** Aviso entre 2.5–3 m; alerta crítica ≤ 1.5 m.
- **Personas:** Aviso temprano con *box ratio* ≥ 0.25 ; alerta crítica si supera 0.35.
- **Huecos y andenes:** Prioridad máxima; alerta inmediata si superan 45% de la imagen.
- **Semáforos:** Detectados, pero sin avisos basados en distancia.

4.9 Modelos y Pipeline de Entrenamiento

- **Detección (implementado):** YOLOv8-small fine-tuned en proyecto-ubaei y exportado a TFLite.
- **Profundidad (implementado):** MiDaS Small en resolución 256–512 px.
- **Segmentación (propuesta):** DeepLabV3+ (no implementada por latencia).
- **Optimización:** Exportación ONNX $\rightarrow$ TFLite parcial.

4.10 Integración

El sistema integrado sigue estos pasos:

1. Captura el frame desde la cámara.
2. YOLOv8-TFLite produce predicciones crudas.
3. Se aplica posprocesamiento: sigmoide, filtros, NMS.
4. MiDaS genera el mapa de profundidad.
5. Se asocia cada detección con la mediana de su profundidad.
6. Se calcula distancia o se usa *box ratio*.
7. Se determina prioridad según clase y cercanía.
8. Se emiten alertas por voz con pyttsx3.

4.11 Diferencias entre la Metodología Inicial y el Sistema Final

El diseño inicial contemplaba segmentación semántica, análisis de vehículos en movimiento y fusión de múltiples métodos de profundidad. Sin embargo, la ejecución en CPU impuso restricciones que llevaron a una arquitectura más ligera.

4.11.1 Cambios Principales

- DeepLabV3+ fue descartado por latencia.
- La fusión MiDaS + pinhole no se integró por complejidad.
- Se reforzó el uso de *box ratio* por su estabilidad.
- Se priorizó YOLOv8-small + TFLite por rendimiento en CPU.

4.11.2 Conclusión

Aunque se descartaron módulos avanzados, la arquitectura final resultó más estable, accesible y funcional para un asistente urbano sin GPU, cumpliendo con el objetivo principal del proyecto.

5 Arquitectura del Sistema y Justificación

El sistema Camino Seguro combina eficiencia, ligereza y modularidad para ejecutarse en tiempo real sin GPU. La arquitectura se divide en tres módulos principales:

5.1 1. YOLOv8 + TensorFlow Lite

YOLOv8-small fue seleccionado por su equilibrio entre precisión y velocidad. Su exportación a TensorFlow Lite permite:

- ejecución fluida en CPU,
- modelo compacto,
- compatibilidad con Android,
- latencia baja y estable.

YOLO es ideal debido a su arquitectura de detección en una sola etapa, que maximiza velocidad y reduce carga computacional.

5.2 2. MiDaS Small para Profundidad Monocular

MiDaS Small permite obtener un mapa de profundidad sin sensores adicionales. Sus ventajas:

- funciona con cualquier cámara RGB,
- opera en CPU,
- mantiene calidad suficiente para obstáculos grandes,
- produce valores estables para distancias relativas.

5.3 3. Módulo de Voz y Lógica de Decisión

Se utiliza pytttsx3 para generar retroalimentación accesible y continua. La lógica del sistema prioriza:

- obstáculos críticos,
- distancias inferiores a 3 m,
- estabilidad gracias a debouncing.

5.4 4. Justificación Global

La arquitectura fue elegida para ser:

- **Ligera:** ejecutable sin GPU.
- **Modular:** detección, voz y profundidad pueden activarse o desactivarse.
- **Escalable:** preparada para migración a Android.
- **Robusta:** fusión de detección + profundidad + tamaño en pantalla.

6 Resultados del Modelo

El modelo YOLOv8 fue entrenado durante 50 épocas utilizando el dataset personalizado *proyecto-ubaei*, compuesto por 10 clases relevantes para la navegación urbana. En esta sección se presentan los resultados cuantitativos y cualitativos obtenidos durante el proceso de entrenamiento y validación del modelo, así como un análisis del desempeño por clase y las métricas globales alcanzadas.

6.1 Rendimiento global del modelo

Durante el entrenamiento, el modelo alcanzó valores estables de mAP y pérdidas, con un comportamiento progresivamente convergente. Las métricas globales observadas fueron:

- **Precisión:** ~ 0.72
- **Recall:** ~ 0.55
- **mAP@0.5:** ~ 0.58
- **mAP@0.5:0.95:** ~ 0.40

Estos valores indican un desempeño adecuado para un modelo ligero destinado a ejecutarse en tiempo real en CPU, especialmente en clases con buena representación en el dataset.

6.2 Distribución del Dataset

La Figura 1 muestra la distribución total de instancias por clase, así como la dispersión espacial de las anotaciones. La clase *Persona* es la dominante del dataset, mientras que clases como *Silla* y *Stop* son significativamente menos frecuentes, lo cual influye directamente en el rendimiento de dichas clases.

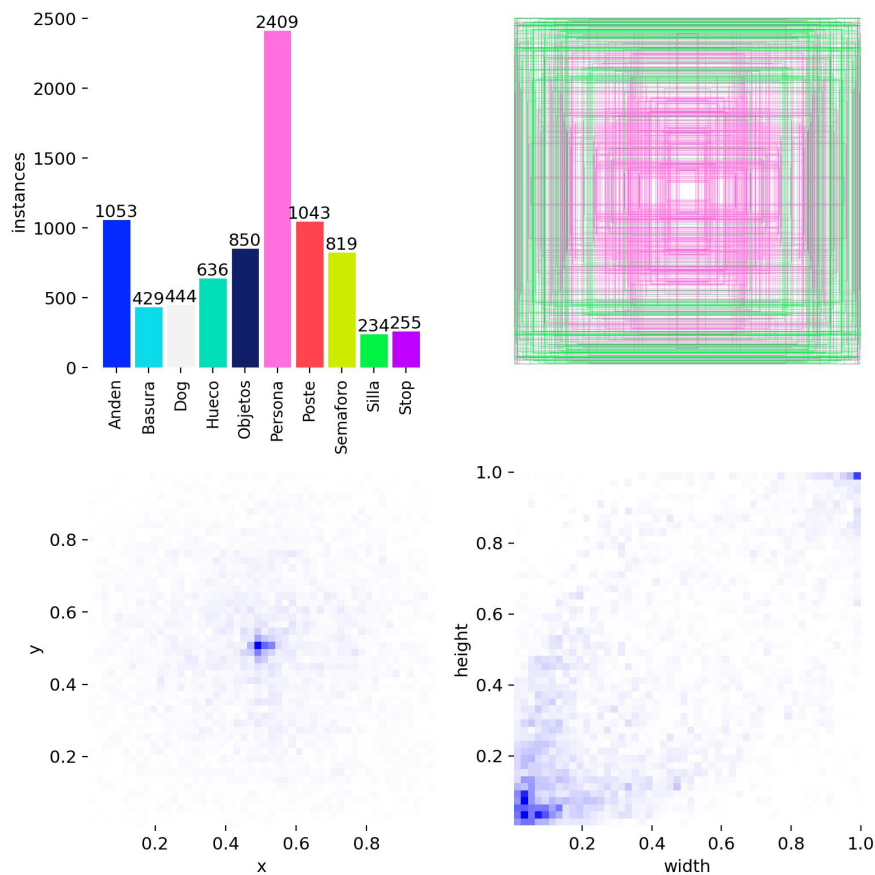


Figure 1: Distribución del dataset por clase y análisis de anotaciones.

6.3 Matriz de confusión

La Figura 2 presenta la matriz de confusión en valores absolutos. Este gráfico permite visualizar cuántas predicciones fueron correctas y en qué clases se concentraron los principales errores. El modelo presenta un excelente desempeño en *Persona*, *Semáforo*, *Dog* y *Poste*, mientras que clases con menor número de ejemplos, como *Basura*, *Stop* y *Silla*, presentan más confusión.

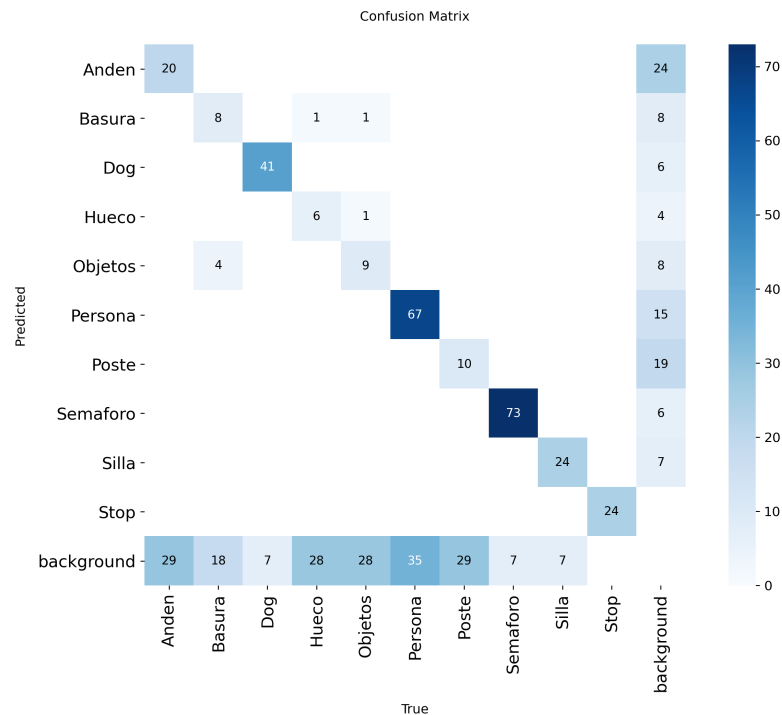


Figure 2: Matriz de confusión del modelo YOLOv8 (valores absolutos).

6.4 Matriz de confusión normalizada

La Figura 3 muestra la matriz de confusión normalizada por clase. Gracias a esto, se observa la proporción real de aciertos independientemente del número de ejemplos presentes por clase. El modelo alcanza valores destacados en:

- **Dog:** 0.85
- **Semáforo:** 0.91
- **Silla:** 0.77
- **Stop:** 1.00

Mientras que clases más ambiguas muestran niveles de confusión mayores:

- **Basura** tiende a confundirse con *Andén*.
- **Objetos** presenta confusión con *Persona*.
- **Hueco** presenta confusión con *Andén* debido a fondos similares en aceras deterioradas.

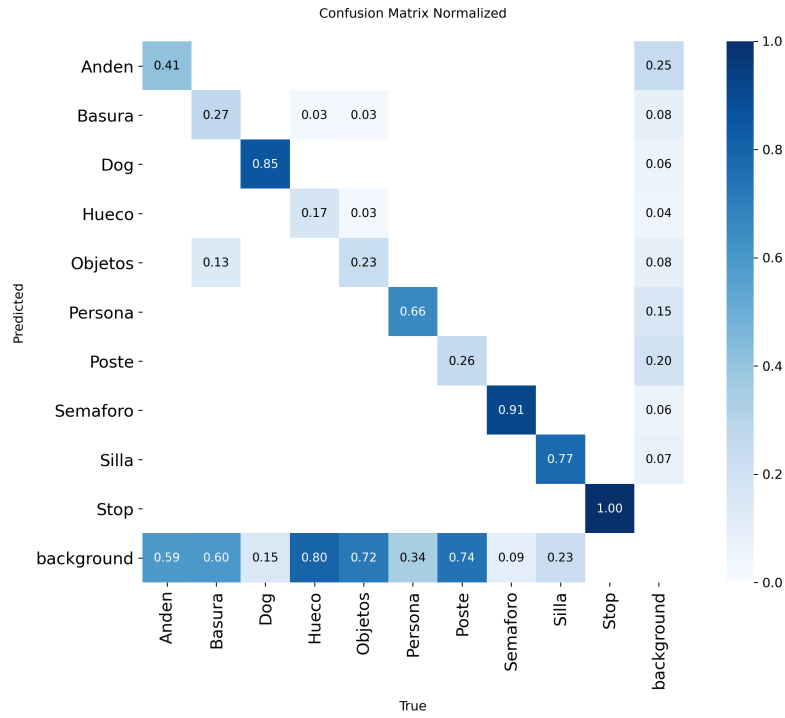


Figure 3: Matriz de confusión normalizada.

6.5 Curvas de entrenamiento y métricas

La Figura 4 muestra la evolución de las pérdidas de entrenamiento y validación, así como las métricas principales (Precisión, Recall y mAP). Se observa un descenso consistente de la pérdida total y un ascenso progresivo del mAP, lo cual indica que el modelo se ajustó correctamente sin presentar sobreajuste significativo.

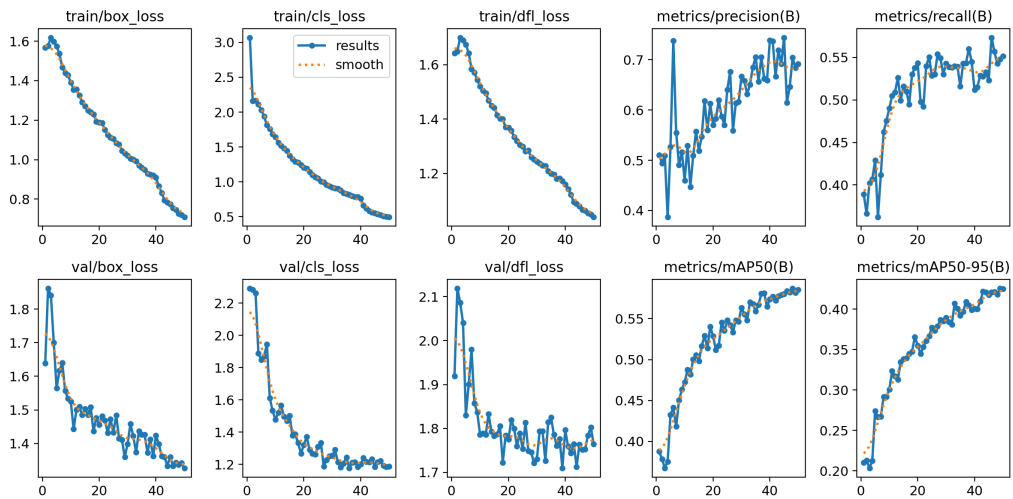


Figure 4: Curvas de pérdida, precisión, recall y mAP durante el entrenamiento.

6.6 Discusión del desempeño

El rendimiento del modelo refleja de manera directa la distribución del dataset:

- Las clases con mayor representación como *Persona*, *Andén* y *Poste* presentan métricas sólidas.
- Las clases escasas, como *Basura*, *Stop* y *Silla*, muestran confusión debido a su baja representación y similitud visual con otras clases del entorno urbano.

- Las métricas globales del modelo son adecuadas para un sistema de asistencia en tiempo real, especialmente al considerar que la versión final del modelo se ejecuta en **TensorFlow Lite** sobre CPU, con restricciones computacionales importantes.

En conjunto, estos resultados confirman que el modelo es capaz de detectar objetos críticos para la navegación urbana, permitiendo su integración efectiva con el módulo de estimación de profundidad y el sistema de alertas por voz implementado.

7 Conclusiones

Camino Seguro demuestra que es posible integrar detección de objetos, estimación de profundidad monocular y retroalimentación auditiva en un sistema accesible y ligero. La arquitectura YOLOv8 + TFLite + MiDaS Small ofrece un equilibrio óptimo entre rendimiento y eficiencia, incluso sin GPU.

Aunque MiDaS no produce distancias métricas exactas, su comportamiento relativo combinado con calibración empírica permite generar alertas útiles para navegación urbana. El sistema final es modular, escalable y funcional, sentando la base para una versión móvil completa en Android.

Como trabajo futuro se propone:

- migrar toda la arquitectura a una aplicación Android mediante CameraX + TFLite,
- mejorar la estimación métrica de distancia,
- implementar segmentación de andenes y huecos,
- incorporar navegación basada en audio espacial.